

Reviewer Pack — Minimal Order of Brauer Groups and SAT Clause Entropy Inequa...

Entry 632d39a39636 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 21:19:25 UTC

Minimal Order of Brauer Groups and SAT Clause Entropy Inequality

Entry ID: 632d39a39636 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 21:19:25 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Brauer Group Theory) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Entropy)

Statement:

For every k -CNF formula φ with m clauses, the minimal order of the Brauer group of the Boolean ring associated with φ , denoted as $|\text{Br}(\text{B}\varphi)|$, is at most a constant factor times the entropy of φ 's clause set, i.e., $|\text{Br}(\text{B}\varphi)| \leq c \cdot H(\varphi)$, where $H(\varphi)$ is the Shannon entropy of φ 's clause set.

Rationale (proposer's reasoning):

The Brauer group captures the algebraic structure of the Boolean ring and provides insights into its linear overapproximations. The entropy of a SAT instance reflects its complexity, and an inequality between these two invariants could expose deep connections between algebraic properties and computational hardness.

Taxonomy category: BrauerGroup_to_SATClauseEntropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0fa1ae00881767d1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the generated k -CNF formulas with m clauses ($n \leq 40$), the ratio $|\text{Br}(\text{B}\varphi)|/H(\varphi)$ is less than or equal to a constant c and does not exceed 10. The conjecture is falsified if any seed produces a ratio greater than 10 or if $\text{support_fraction} < 0.8$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Brauer group AND Boolean satisfiability - title:CNF formula AND Brauer group order AND entropy - arxiv:algebra q-brauer-group AND sat entropies

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_k_cnf(k: int, m: int) -> list:
    variables = set(range(1, k + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, k)
        clauses.append(clause)
    return clauses

def incidence_matrix(clauses: list) -> list:
    n = len(clauses)
    m = max(len(c) for c in clauses)
    matrix = [[0] * m for _ in range(n)]
    for i, clause in enumerate(clauses):
        for j, var in enumerate(clause):
            matrix[i][j] = 1
    return matrix

def determinant(matrix: list) -> int:
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        sign = (-1) ** j
        det += sign * matrix[0][j] * determinant(submatrix)
    return det

def shannon_entropy(clauses: list) -> float:
    n = len(clauses)
```

```

m = max(len(c) for c in clauses)
freqs = [sum(1 for c in clauses if i in c) / n for i in range(1, m + 1)]
entropy = -sum(p * math.log2(p) for p in freqs if p > 0)
return entropy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    k = random.randint(3, 5) # Number of variables
    m = random.randint(k + 1, 2 * k + 1) # Number of clauses
    n = len(set(abs(v) for v in generate_k_cnf(k, m)))

    if n < 5:
        return {
            "metric_name": "Brauer Group Order / Clause Entropy Ratio",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "n_min=5 required"
        }

    clauses = generate_k_cnf(k, m)
    matrix = incidence_matrix(clauses)
    det = determinant(matrix)
    entropy = shannon_entropy(clauses)

    if entropy == 0:
        return {
            "metric_name": "Brauer Group Order / Clause Entropy Ratio",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "Entropy is zero"
        }

    ratio = abs(det) / entropy
    return {
        "metric_name": "Brauer Group Order / Clause Entropy Ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": ratio <= 10,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

if not all(r["instances_tested"] > 0 for r in results):
    print("RESULT: INCONCLUSIVE insufficient_data")
    sys.exit(0)

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
support_count = sum(1 for r in results if r["conjecture_holds"])
support_fraction = support_count / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=NA support_fraction={support_fraction}")
elif any(r["metric_value"] > 10 for r in results):
    first_failing_seed = next(seed for seed, r in enumerate(results) if r["metric_value"] > 10)
    print(f"RESULT: FALSIFIED counterexample=\"Ratio exceeds 10\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE support_fraction < 0.8")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94ee3c97.py", line 100, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94ee3c97.py", line 57, in run_trial
    n = len(set(abs(v) for v in generate_k_cnf(k, m)))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94ee3c97.py", line 22, in generate_k_cnf
    clause = random.sample(variables, k)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. "
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the error in the test code to allow for proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12048
2	preregistration	ollama_remote	glm4:latest	0	0	9988
3	novelty	ollama_remote	glm4:latest	0	0	8313
4	novelty	ollama_remote	glm4:latest	0	0	9167
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37727
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13071
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12432
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11516
9	judge	ollama_remote	glm4:latest	0	0	16442

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 130704 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/632d39a39636.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/632d39a39636.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/632d39a39636.tar.gz` (if generated)